

IoT Platforms: ESP32 Programming Options

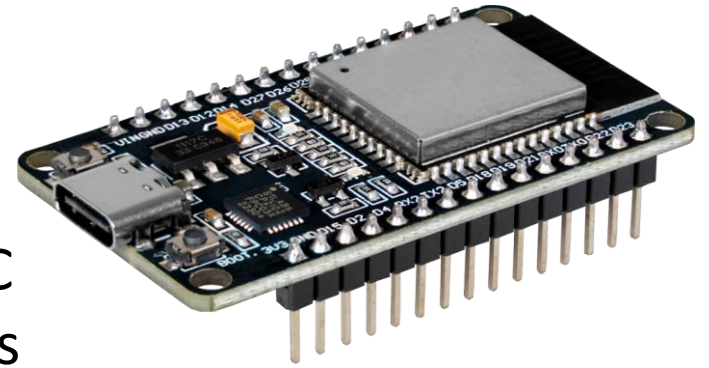
Dr. Hiran Ekanayake

University of Colombo School of Computing

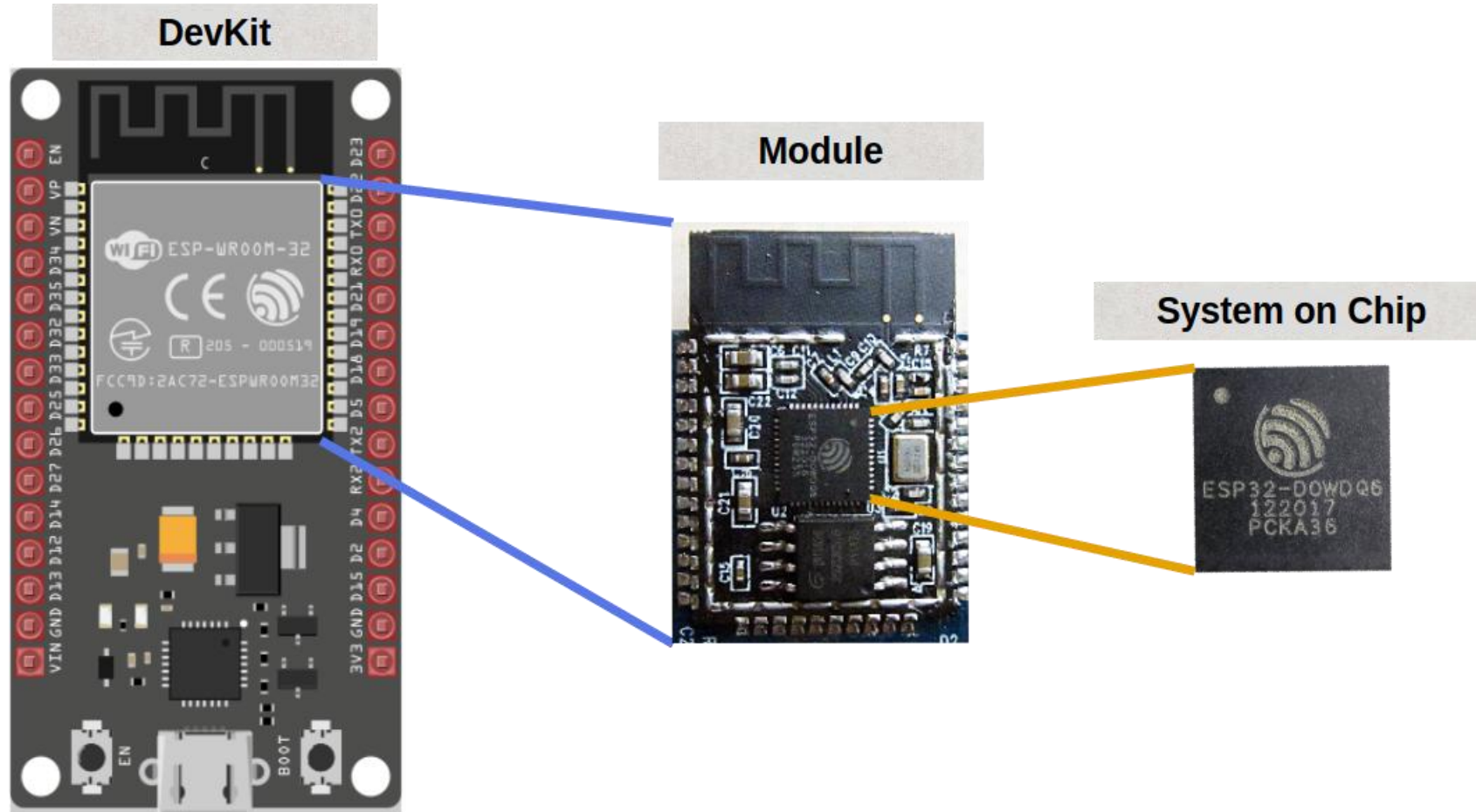
2026

What is ESP32?

- The **ESP32** is a low-cost, low-power microcontroller chip made by [Espressif Systems](https://www.espressif.com/). It's widely used in electronics, robotics, IoT (Internet of Things), and embedded systems projects because it has built-in **Wi-Fi** and **Bluetooth**.
- Features:
 - Built-in **Wi-Fi** and **Bluetooth**
 - Can be used in industry applications, -40°C to $+125^{\circ}\text{C}$
 - Low power consumption for battery-powered devices
 - Highly-integrated with in-built antenna switches, RF balun, power amplifier, low-noise receive amplifier, filters, etc.
 - Programmable using Arduino IDE, PlatformIO, MicroPython, ESP-IDF (Espressif's official framework), etc.

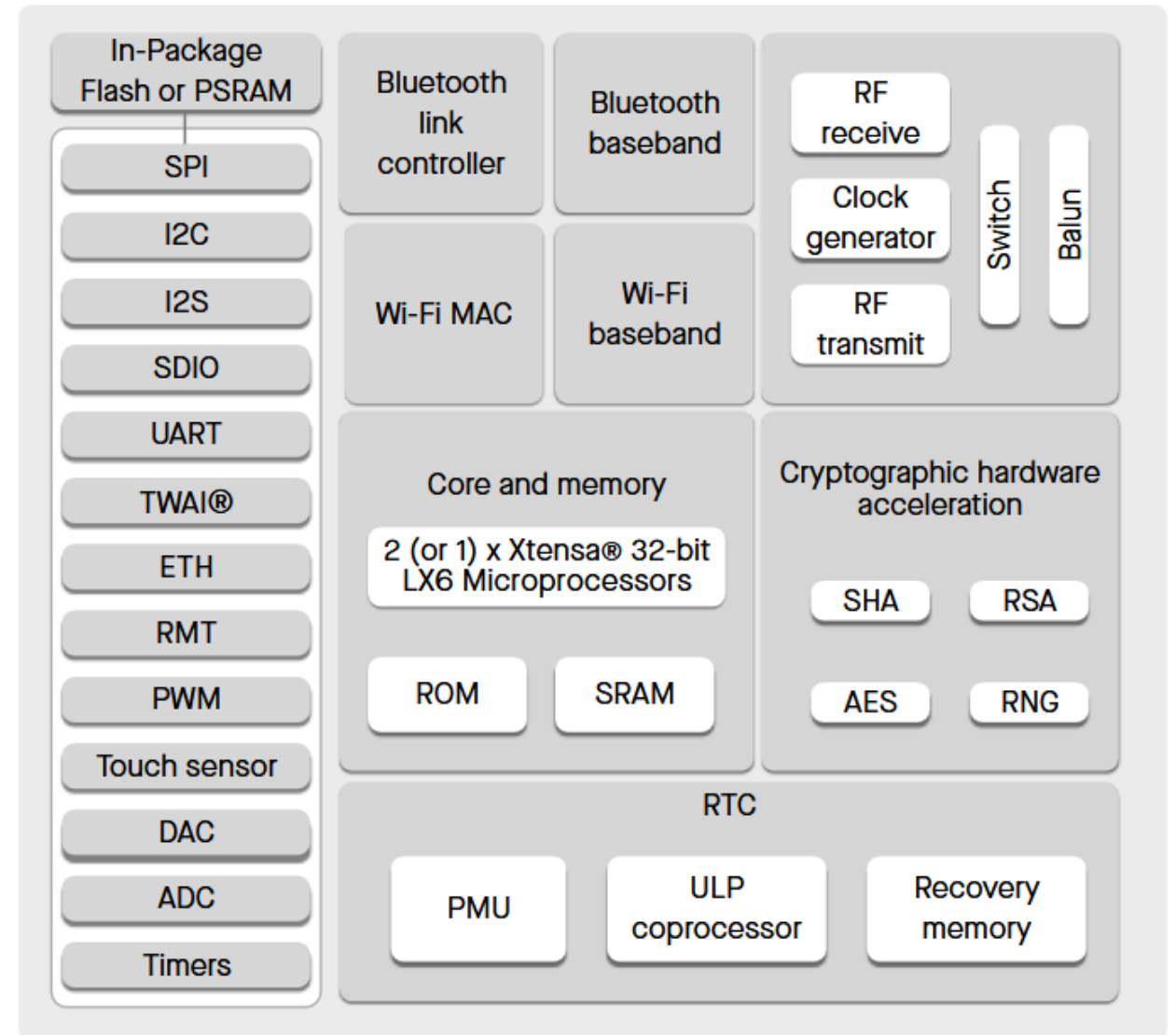


ESP32 Variants: System on Chip (SoC), Module, and Development Boards (Devkits)



System on Chip (SoC)

- The core and most fundamental form of the ESP32. It integrates essential components of an electronic system or computer into a single Integrated Circuit (IC).
- E.g., ESP32-D0WDQ6-V3
 - D: Dual Core
 - 0: No in-package flash
 - WD: Wi-Fi b/g/n + Bluetooth/Bluetooth LE dual mode
 - Q6: QFN 6*6 package
 - V3: Chip revision v3.0



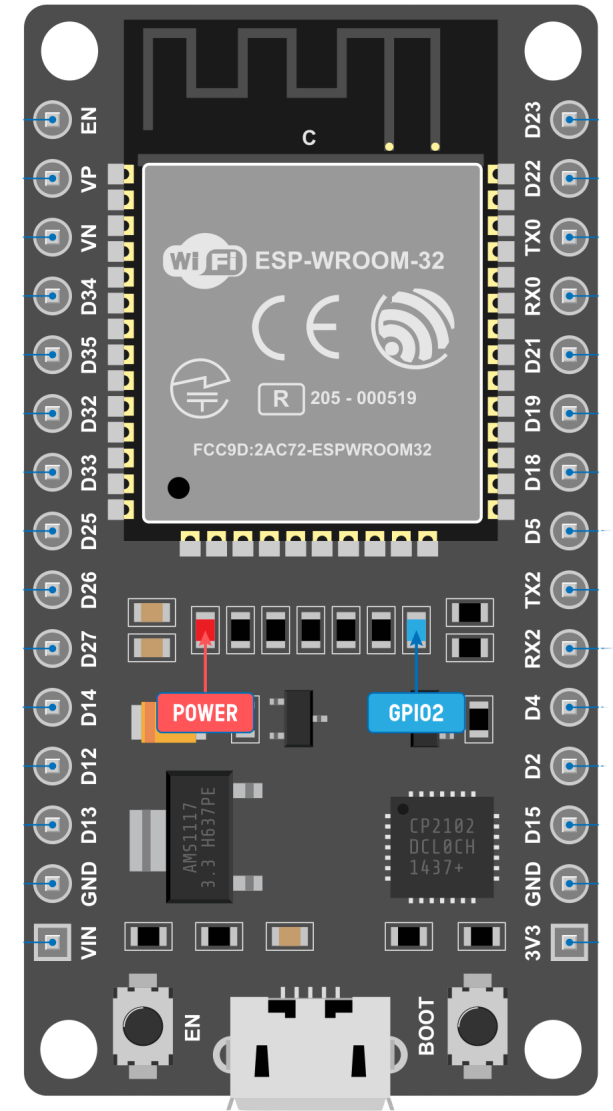
Module

- Modules build upon the SoC and provide a more user-friendly, ready-to-use solution.
 - E.g., ESP32-WROOM-32
- Why choose a module?
 - **Pre-Certification:** Modules are pre-certified for regulatory compliance, saving time and effort.
 - **Integrated Components:** They include PCB antennas, a crystal oscillator, and a flash memory chip.
 - **Shielded Design:** Modules come with an EMI shield to reduce electromagnetic interference.



Development Boards (Devkits)

- Devkits simplify the use of ESP32 modules for prototyping and development. They include additional components to make the ESP32 module easier to work with for beginners and experienced developers alike.
 - E.g., **ESP32 DevKit v1**, **ESP32-S3-DevkitC**
- Key features:
 - **USB Interface**: Enables easy programming and debugging.
 - **Voltage Regulators**: Provides stable operation for the ESP32.
 - **Pin Breakouts**: Makes ESP32 pins accessible for connecting external components like sensors and displays.
 - **Boot and Reset Buttons**: Provides control over module operations.



ESP32 Product Selector

<https://products.espressif.com/#/product-selector>

- 312 ICs/Modules & 80 Development Boards (June 2026)

Wi-Fi ?

Bluetooth ?

Temperature ?

Thread/Zigbee ?

Products ?

Series ?

Status ?

Core ?

Antenna ?

Package ?

Memory ?

Peripherals ?

List: 312 items



IC/Module



Development Board

☐	Index	PN	Name ↕	Marketing Status	Replaced by	Type ↕	Wi-Fi
☐	1	ESP32-S3	ESP32-S3	Mass Production	--	SoC	IEEE 802.11b/g/n; 2.4 GH...
☐	2	ESP32-E22	ESP32-E22	Sample	--	SoC	N/A
☐	3	ESP32-E22-M2-1-KAE	ESP32-E22-M2-1	Sample	--	Module	IEEE 802.11b/g/n; 2.4/5/...
☐	4	ESP32-S3R2	ESP32-S3	EOL	ESP32-S3RH2	SoC	IEEE 802.11b/g/n; 2.4 GH...
☐	5	ESP32-S3RH2	ESP32-S3	Mass Production	--	SoC	IEEE 802.11b/g/n; 2.4 GH...
☐	6	ESP32-S3D8	ESP32-S3	Mass Production	--	SoC	IEEE 802.11b/g/n; 2.4 GH...

List: 80 items



IC/Module



Development Board

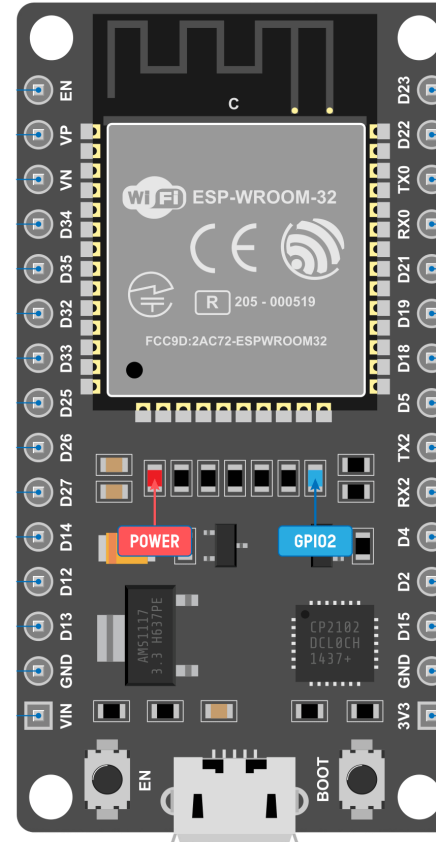
Index	PN	Name ↕	Marketing Status	Replaced by	Temp (°C) ↕	Flash (MB) ↕	PSRAM (MB) ↕
1	ESP32-S2-Saola-1R	ESP32-S2-Saola-1	EOL	--	-40 ~ 85	4	2
2	ESP32-C6-DevKitC-1-N8	ESP32-C6-DevKitC-1	Mass Production	--	-40 ~ 85	8	N/A
3	ESP32-H2-DevKitM-1-N4	ESP32-H2-DevKitM-1	Replaced	ESP32-H2-D...	-40 ~ 85	4	

ESP32 Families

Series	Main Focus	Wireless	CPU Architecture	Typical Applications
ESP32 (Original)	General-purpose IoT	Wi-Fi + Bluetooth	Xtensa	IoT, robotics, automation
ESP32-S Series	AI, USB, multimedia	Wi-Fi + Bluetooth	Xtensa/RISC-V (depending on model)	HMI, audio, vision
ESP32-C Series	Cost-optimized connectivity	Wi-Fi + BLE	RISC-V	Sensors, smart devices
ESP32-H Series	Matter/Thread/Zigbee	IEEE 802.15.4	RISC-V	Smart home hubs and nodes
ESP32-P Series	High-performance AI/edge computing	Wi-Fi + BLE	RISC-V	AI and advanced edge devices

Original ESP32 Series

- The original ESP32 introduced:
 - Dual-core Xtensa CPU
 - Wi-Fi and Bluetooth Classic/BLE
 - Rich GPIO support
 - ADC, DAC, PWM, SPI, I²C, UART
- Popular development boards:
 - ESP32 DevKit V1, NodeMCU-32S, WROOM-32
- Typical uses:
 - IoT projects, Robotics, Home automation, Industrial monitoring



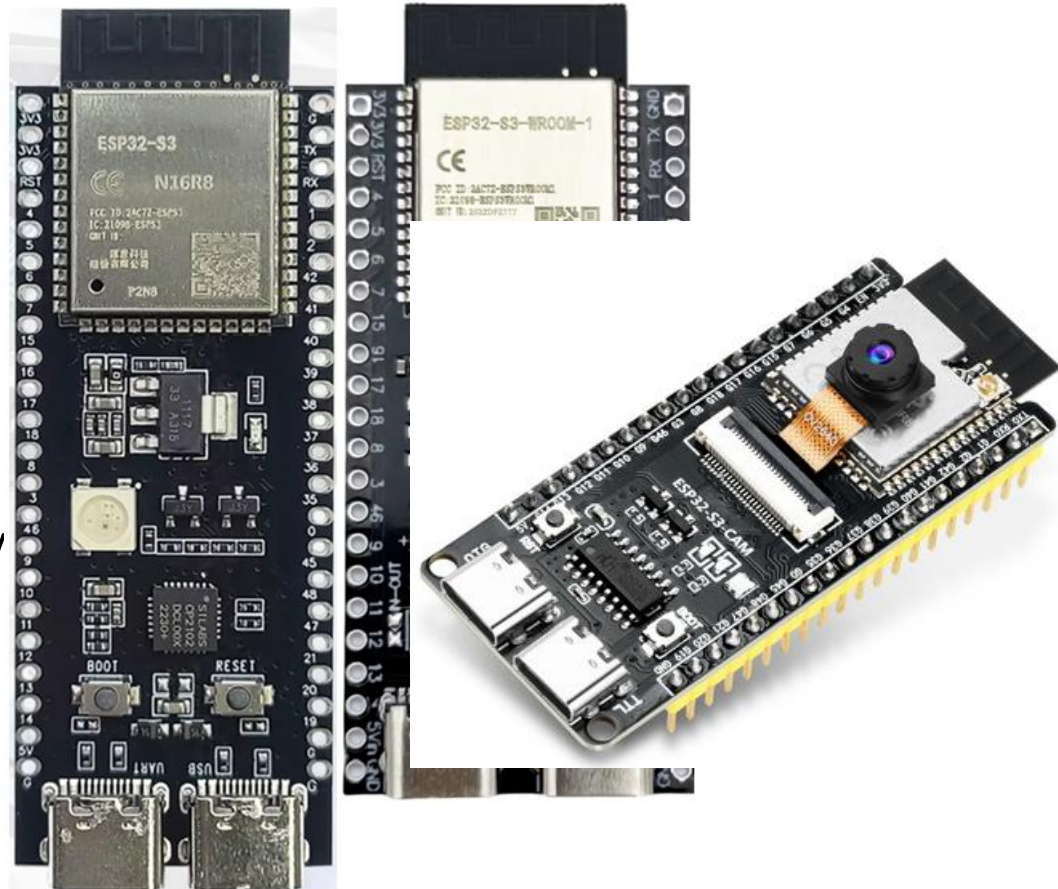
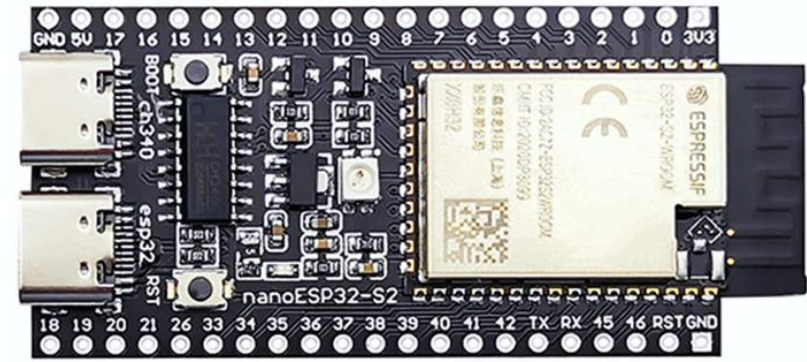
ESP32 DevKit V1 vs Arduino Uno R3

Arduino Uno R3	ESP32 DevKit V1
Learning embedded programming	Internet of Things (IoT)
LED and button experiments	Smart home devices
Sensor interfacing	Cloud-connected sensors
Motor control	MQTT and HTTP applications
Basic robotics	Bluetooth applications
Data acquisition	Web servers
Classroom laboratories	AI/ML edge applications
Control systems	Wireless robotics

Feature	ESP32 DevKit V1	Arduino Uno R3
Microcontroller	ESP32 (Xtensa LX6)	ATmega328P
CPU Architecture	32-bit	8-bit AVR
CPU Cores	Dual-core	Single-core
Clock Speed	Up to 240 MHz	16 MHz
SRAM	520 KB	2 KB
Flash Memory	Typically 4 MB	32 KB (0.5 KB bootloader)
EEPROM	Emulated in Flash	1 KB
Operating Voltage (Logic)	3.3 V	5 V
Input Voltage (Board)	5 V via USB or VIN	7–12 V recommended (VIN), 5 V via USB
Digital GPIO	Up to 34 usable	14
Analog Inputs	Up to 18 (12-bit ADC)	6 (10-bit ADC)
Analog Outputs	2 DAC channels (8-bit)	None (PWM only)
PWM Channels	Up to 16	6
UART	3	1
SPI	4	1
I ² C	2	1
I ² S	2	Not available
CAN (TWAI)	Yes	No
Capacitive Touch	10 touch pins	No
Hall Sensor	Yes (original ESP32)	No
Wi-Fi	Built-in 2.4 GHz 802.11 b/g/n	Not built-in
Bluetooth	Bluetooth Classic + BLE	Not built-in
USB Programming	Micro-USB (CP2102/CH340 on most boards)	USB Type-B (ATmega16U2 or CH340 on clones)
Deep Sleep	Yes (≈5 µA typical)	Limited power-saving modes
Programming Languages	Arduino, ESP-IDF, MicroPython, CircuitPython	Arduino (C/C++), AVR-GCC
Typical Cost	Low (often comparable to or less than Uno clones)	Moderate

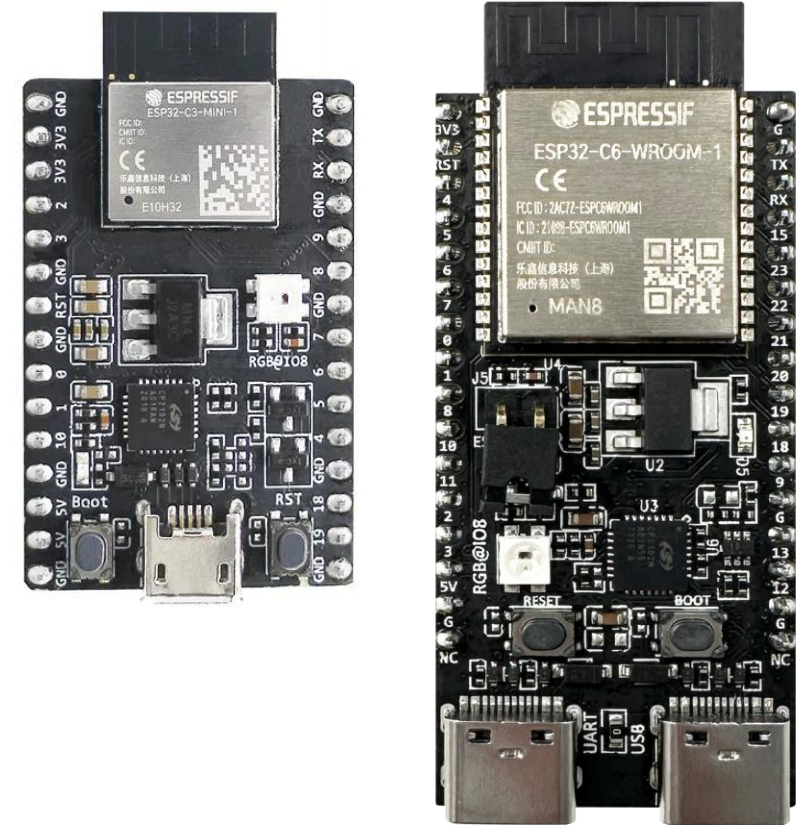
ESP32-S Series (Feature-Rich and Multimedia)

- The "S" generally stands for **enhanced peripherals and multimedia capabilities**.
- ESP32-S2:
 - Features: Single-core Xtensa CPU, Wi-Fi only (no Bluetooth), Native USB OTG (Host + Device), Improved security, More GPIOs
 - Applications: USB devices, Data loggers, Human-machine interfaces
- **ESP32-S3:**
 - Features: Dual-core Xtensa LX7, Wi-Fi + Bluetooth LE, Vector instructions for AI/ML, USB OTG, Larger memory support
 - Applications: Voice assistants, Camera projects, Edge AI, Computer vision
 - Popular boards: ESP32-S3 DevKit, ESP32-S3-CAM
- **ESP32-S31 – AI IoT Platform**



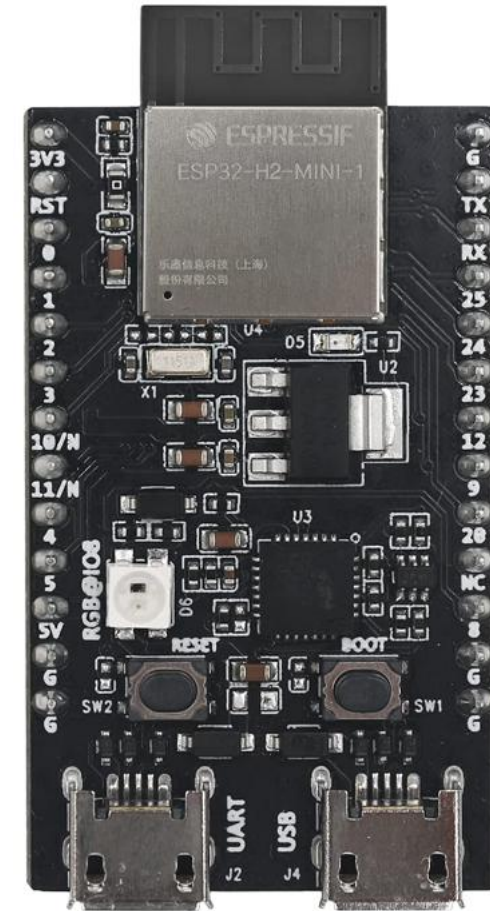
ESP32-C Series (Cost-Optimized Connectivity)

- The "C" series emphasizes **low cost and low power consumption**.
- **ESP32-C2:**
 - Features: Single-core RISC-V, Wi-Fi 4, Bluetooth LE, Very low cost
 - Applications: Smart plugs, Sensors, Entry-level IoT devices
- **ESP32-C3:**
 - Features: Single-core 32-bit RISC-V, Wi-Fi + BLE 5, Enhanced security, Low power
 - Applications: IoT nodes, Smart appliances, Wearables
- **ESP32-C6:**
 - Features: RISC-V core, Wi-Fi 6, Bluetooth 5, Thread support, Zigbee support
 - Applications: Matter devices, Smart homes, Industrial IoT
- **ESP32-C5:**
 - Features: Dual-band Wi-Fi 6 (2.4 GHz and 5 GHz), Bluetooth LE, RISC-V architecture
 - Applications: High-performance wireless IoT



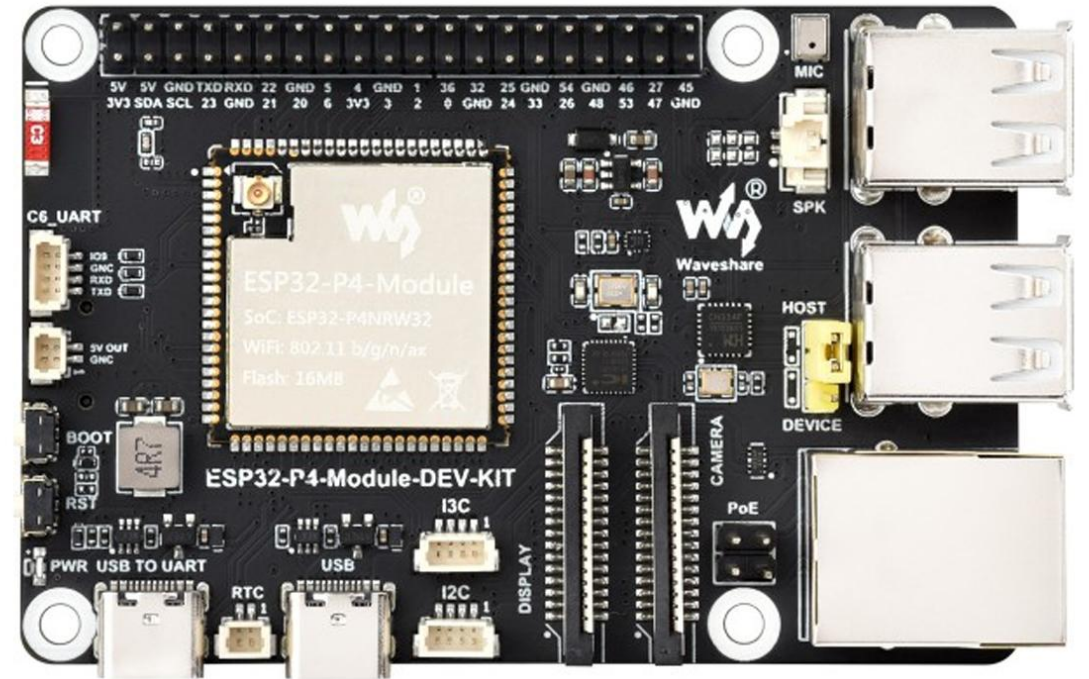
ESP32-H Series (Smart Home Connectivity)

- The "H" series focuses on **IEEE 802.15.4 networking technologies**.
- ESP32-H2:
 - Features: RISC-V CPU, Zigbee, Thread, Bluetooth LE, **No Wi-Fi**
 - Applications: Matter-over-Thread devices, Smart switches, Smart lighting, Home automation sensors
 - The H2 is designed to act as a low-power smart-home node rather than a Wi-Fi device.



ESP32-P Series (High-Performance Edge AI)

- ESP32-P4:
 - Features: High-performance dual-core RISC-V, Large memory support, Advanced graphics capabilities, Camera and display interfaces, AI acceleration support, **No integrated Wi-Fi or Bluetooth, Often paired with wireless chips such as C6 or H2**
 - Applications: Edge AI, Industrial HMI, Smart displays, Vision systems



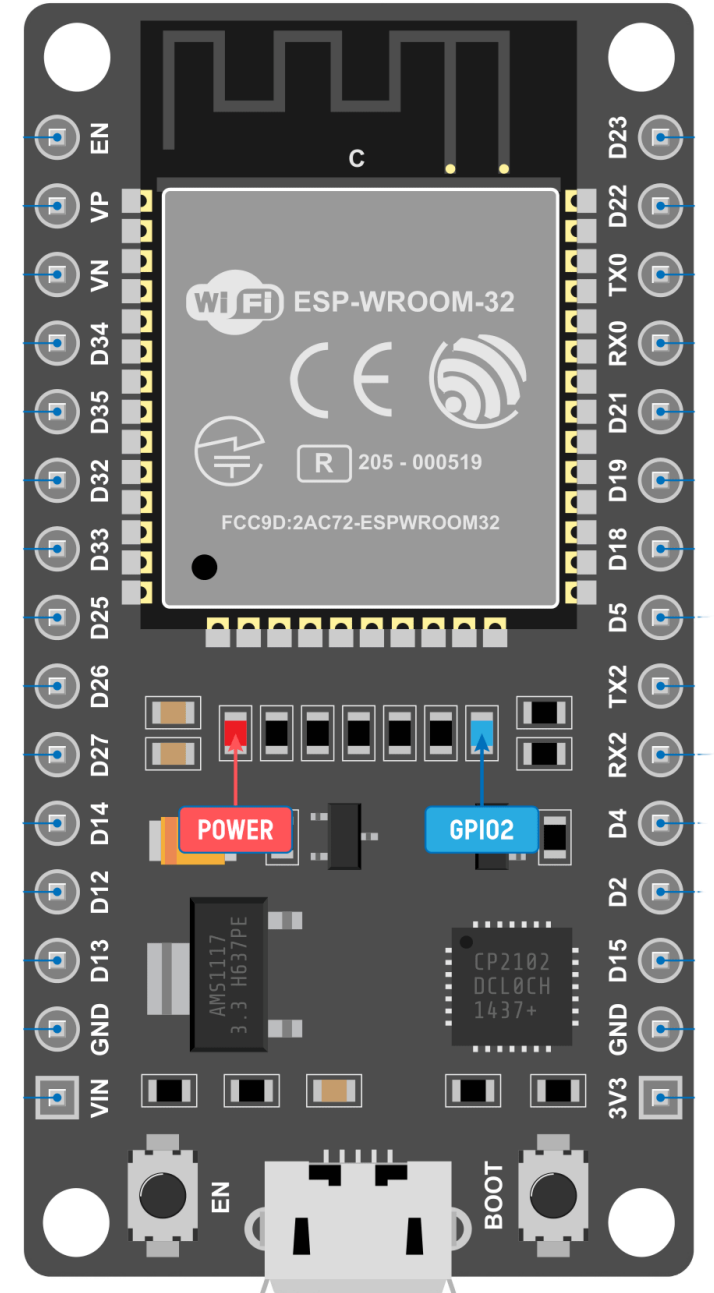
Choosing the Right ESP32

If you want...	Recommended Variant	SL Availability (2026)
General IoT learning	ESP32 DevKit (original)	Rs. 1,400/=
Modern IoT + AI + USB	ESP32-S3	ESP32-S3 DevKitC-1 Rs. 1,850/=; ESP32-S3 Nano Dev Board 3,650/=
Lowest-cost IoT nodes	ESP32-C3	SuperMini ESP32-C3 Dev Board Rs. 780/=
Matter/Zigbee projects	ESP32-C6 or H2	ESP32-C6 Dev Board Rs. 2,250/=
Computer vision	ESP32-S3	
Industrial HMI and Edge AI	ESP32-P4	

mpn	内置sram	内置rom	外扩psram	外扩flash
ESP32-S3-WROOM-1-N4	512K	384K	0M	4M
ESP32-S3-WROOM-1-N8	512K	384K	0M	8M
ESP32-S3-WROOM-1-N16	512K	384K	0M	16M
ESP32-S3-WROOM-1-N4R8	512K	384K	8M	4M
ESP32-S3-WROOM-1-N4R2	512K	384K	2M	4M
→ ESP32-S3-WROOM-1-N8R2	512K	384K	2M	8M
ESP32-S3-WROOM-1-N16R2	512K	384K	2M	16M
→ ESP32-S3-WROOM-1-N8R8	512K	384K	8M	8M
→ ESP32-S3-WROOM-1-N16R8	512K	384K	8M	16M

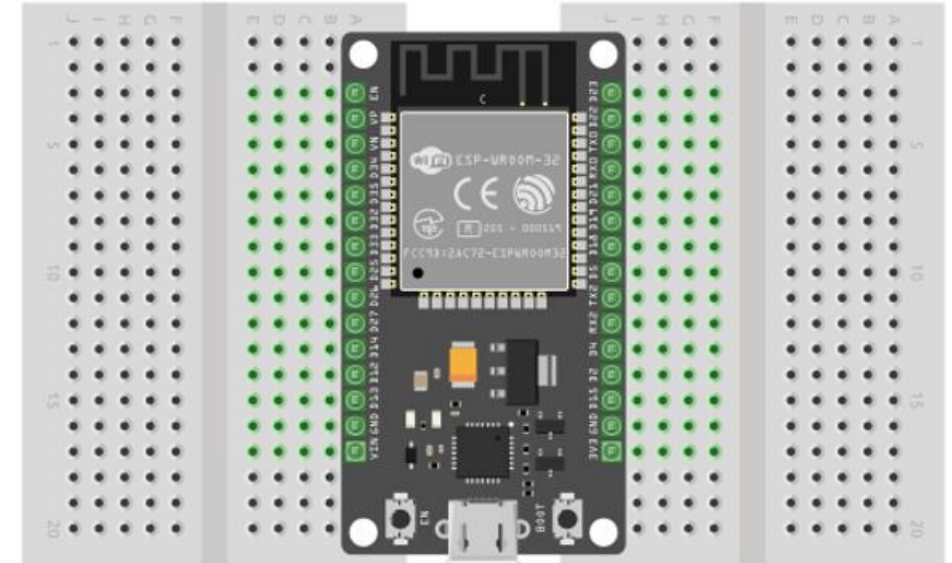
ESP32 DevKit V1

DOIT-ESP32-DevKit-V1



Why start with “ESP32 DevKit V1”?

- It is a development board that typically contains an **ESP32-WROOM-32 module** (ESP32-D0WD SoC) and exposes most GPIO pins for easy experimentation.
- Easy USB Programming
 - Simply connect a USB cable and upload code from Arduino IDE. No external programmer is required.
- Breadboard-Friendly Prototyping
- Massive Community Support
- Low Cost
- Stable and Mature Ecosystem
 - Most Arduino libraries were originally tested on these modules.

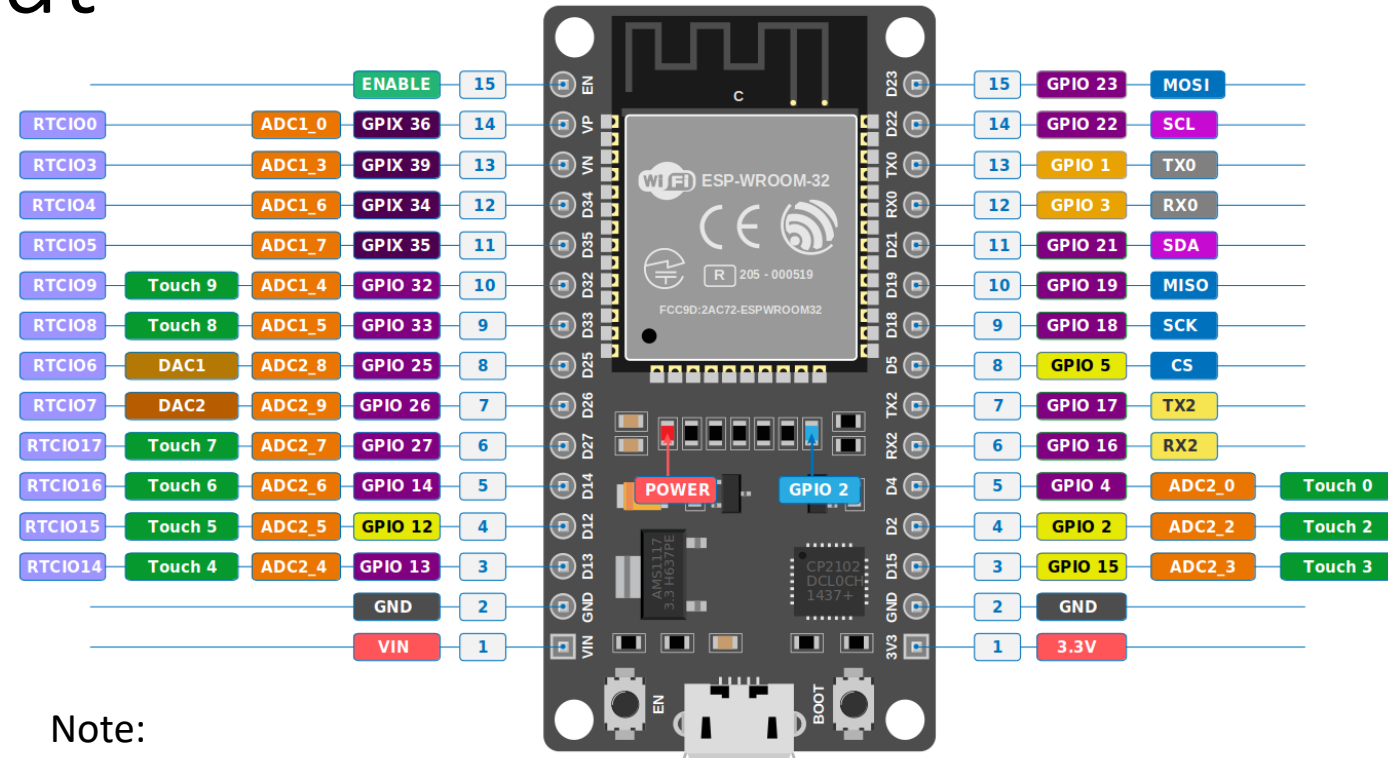


Official Documents

- [ESP32 Datasheet](#) (for developers)
 - Provides a high-level description (features) of the ESP32 chip.
 - Use it when asking: How much RAM does ESP32 have? How many UARTs are available? What is the operating voltage? What wireless standards are supported? Etc.
- [ESP32 Technical Reference](#) (for advanced developers)
 - Explains how the internal hardware actually works.
 - Use it when: Writing low-level drivers, Using ESP-IDF directly, Studying computer architecture, Understanding hardware internals, etc.
- [ESP32-WROOM32-Datasheet](#) (for hardware designers)
 - Describes the ESP32-WROOM-32 module (not the chip).
 - Use it when designing a custom IoT product such as: Smart thermostat, Industrial sensor node, Smart agriculture controller, etc.

ESP32 DevKit V1 (Development Board) Specification & Pinout

Feature	Specification
Main Module	ESP32-WROOM-32
CPU	Dual-core Xtensa LX6
Clock Speed	Up to 240 MHz
SRAM	520 KB
Flash Memory	Typically 4 MB
Wi-Fi	IEEE 802.11 b/g/n (2.4 GHz)
Bluetooth	Bluetooth v4.2 BR/EDR + BLE
Operating Voltage	3.3 V
USB Interface	Micro-USB
USB-UART Chip	CP2102 or CH340 (board dependent)
GPIO Pins	Up to 34 usable GPIOs (30, 36, etc.)
ADC	18 channels, 12-bit
DAC	2 channels, 8-bit
PWM	Up to 16 channels
UART	3
SPI	4
I ² C	2
I ² S	2
Touch Sensors	10 capacitive touch pins
Deep Sleep Current	~5 μ A
On-board Buttons	EN (Reset), BOOT



Note:

GPIO 1 (Tx) and GPIO 3 (Rx) – flashing and debugging only.

Built-in blue LED connected to GPIO2

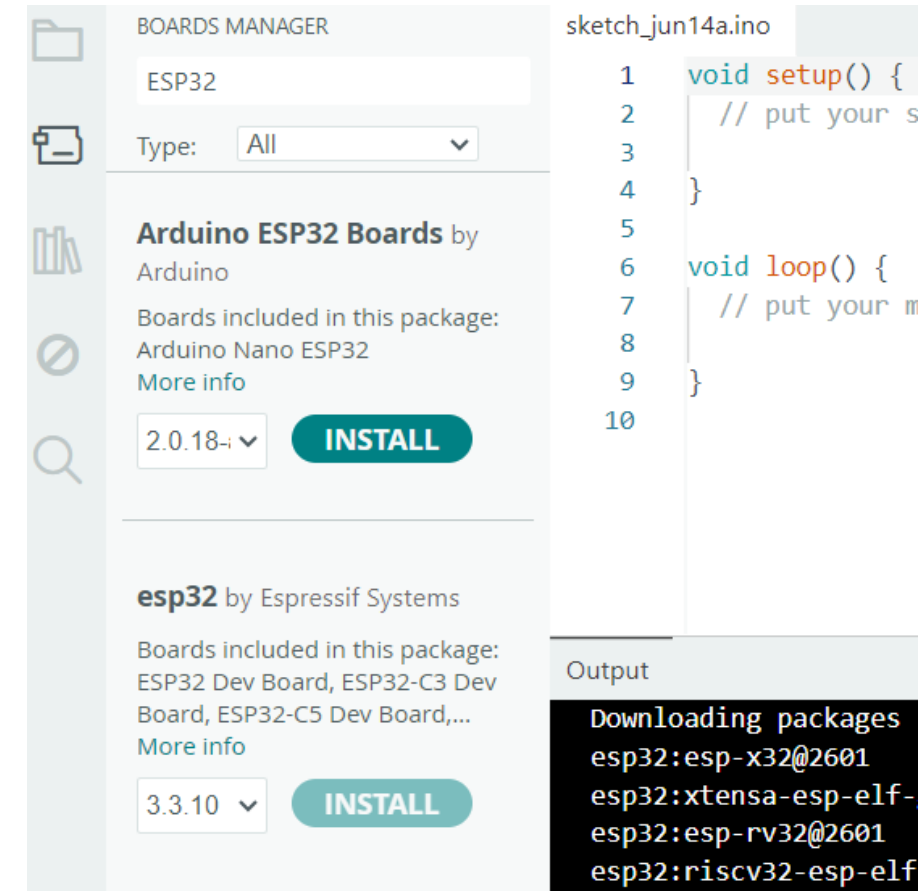
How to Program the ESP32?

- Arduino C/C++ using the Arduino core for the ESP32
- VS Code + PlatformIO IDE
- Espressif IDF (IoT Development Framework)
- Micropython
- JavaScript
- LUA
- Etc.

Programming ESP32 DevKit V1 with **Arduino IDE** on Windows 10/11 (64-bit)

Steps

- **Install Arduino IDE** (assumed Windows 10/11 64-bit, IDE 2.3.10 as of June 2026)
- **Install ESP32 Board Support Package:**
 - File → Preferences
 - In Additional Boards Manager URLs, add: https://espressif.github.io/arduino-esp32/package_esp32_index.json → OK
 - Tools → Board → Boards Manager
 - Search for “ESP32” and install “ESP32 by Espressif Systems”
- **Connect ESP32 DevKit V1:**
 - Windows usually installs drivers automatically. If the COM port does not appear, install the appropriate driver manually.



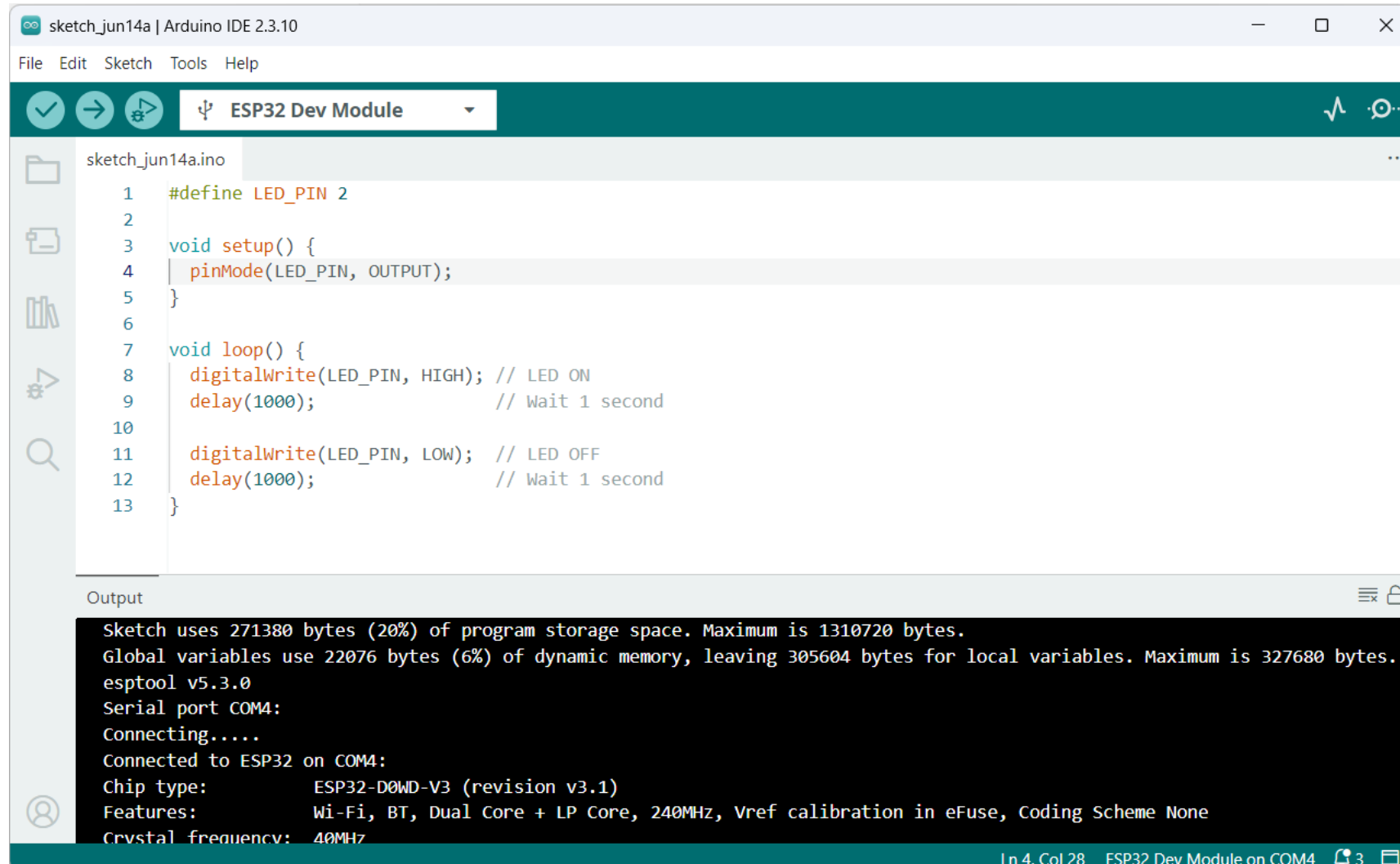
Steps Cont.

- **Select the Board:**
 - Tools → Board → ESP32 Arduino
 - Choose “DOIT ESP32 DEVKIT V1” or “ESP32 Dev Module”
- **Select the COM Port:**
 - Tools → Port
 - Select the COM port assigned to ESP32.
- Create Your Program
- **Upload the Program:**
 - Click “Upload”
 - If upload fails: Hold the **BOOT** button. Click **Upload**. Release BOOT when uploading starts.

Missing USB-UART driver

- Check Device Manager:
 - Windows Key + X → Device Manager
 - Do you see anything like:
 - Silicon Labs CP210x USB to UART Bridge
 - USB Serial CH340
 - USB Serial Device
- Install the driver:
 - CP2102 Driver → Silicon Labs CP210x Driver Downloads
 - CH340 Driver → WCH CH340 Driver Downloads
- After installation: Reboot Windows. Reconnect ESP32. Check Device Manager again.

Program 01: Blink (Built-in) LED



sketch_jun14a | Arduino IDE 2.3.10

File Edit Sketch Tools Help

ESP32 Dev Module

sketch_jun14a.ino

```
1  #define LED_PIN 2
2
3  void setup() {
4    pinMode(LED_PIN, OUTPUT);
5  }
6
7  void loop() {
8    digitalWrite(LED_PIN, HIGH); // LED ON
9    delay(1000);                // Wait 1 second
10
11    digitalWrite(LED_PIN, LOW);  // LED OFF
12    delay(1000);                // Wait 1 second
13  }
```

Output

Sketch uses 271380 bytes (20%) of program storage space. Maximum is 1310720 bytes.
Global variables use 22076 bytes (6%) of dynamic memory, leaving 305604 bytes for local variables. Maximum is 327680 bytes.
esptool v5.3.0
Serial port COM4:
Connecting.....
Connected to ESP32 on COM4:
Chip type: ESP32-D0WD-V3 (revision v3.1)
Features: Wi-Fi, BT, Dual Core + LP Core, 240MHz, Vref calibration in eFuse, Coding Scheme None
Crystal frequency: 40MHz

Ln 4, Col 28 ESP32 Dev Module on COM4 3

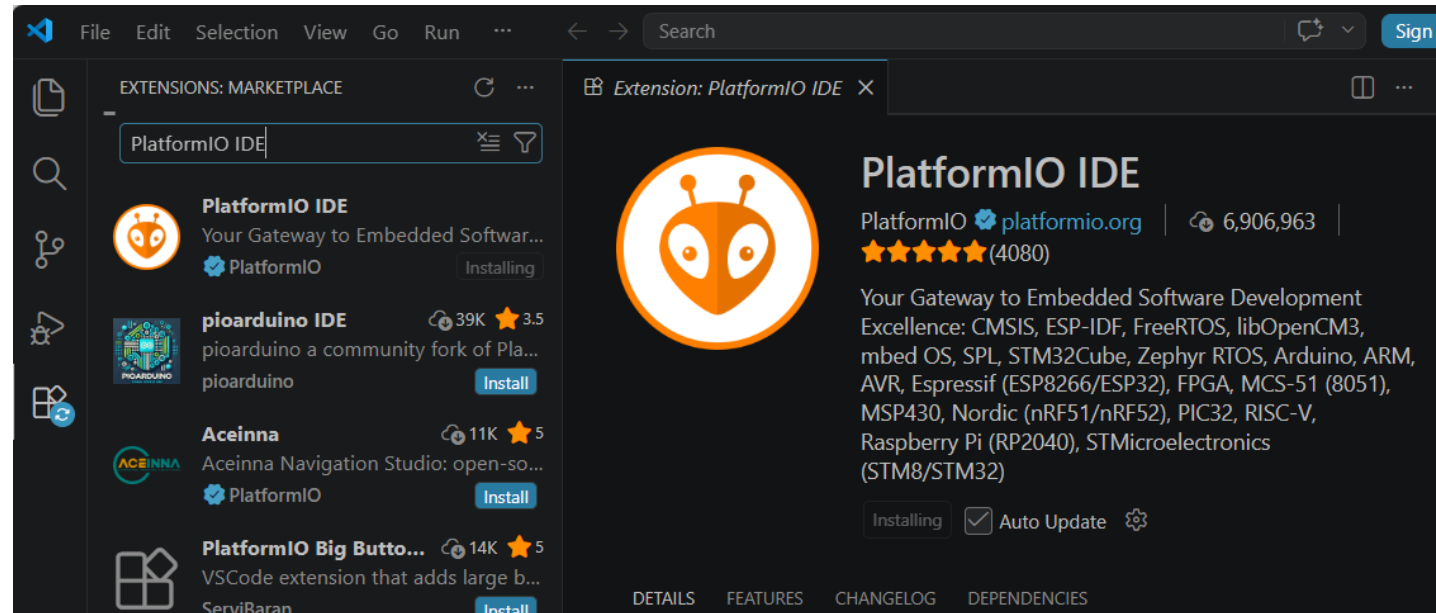
Programming ESP32 DevKit V1 with VS Code + PlatformIO on Windows 10/11 (64-bit)

What is PlatformIO?

- **PlatformIO** is an open-source ecosystem for embedded systems development that runs inside **Visual Studio Code (VS Code)**.
- Instead of being limited to ESP32, it supports many platforms: ESP32, ESP8266, Arduino AVR, STM32, Raspberry Pi Pico, Nordic nRF52, Teensy, ARM Cortex-M boards.
- While the Arduino IDE is excellent for beginners, **PlatformIO** is often preferred for university courses, professional development, and larger projects because it provides a modern development environment, better project organization, library management, unit testing, debugging support, and integration with version control systems such as Git.

Steps

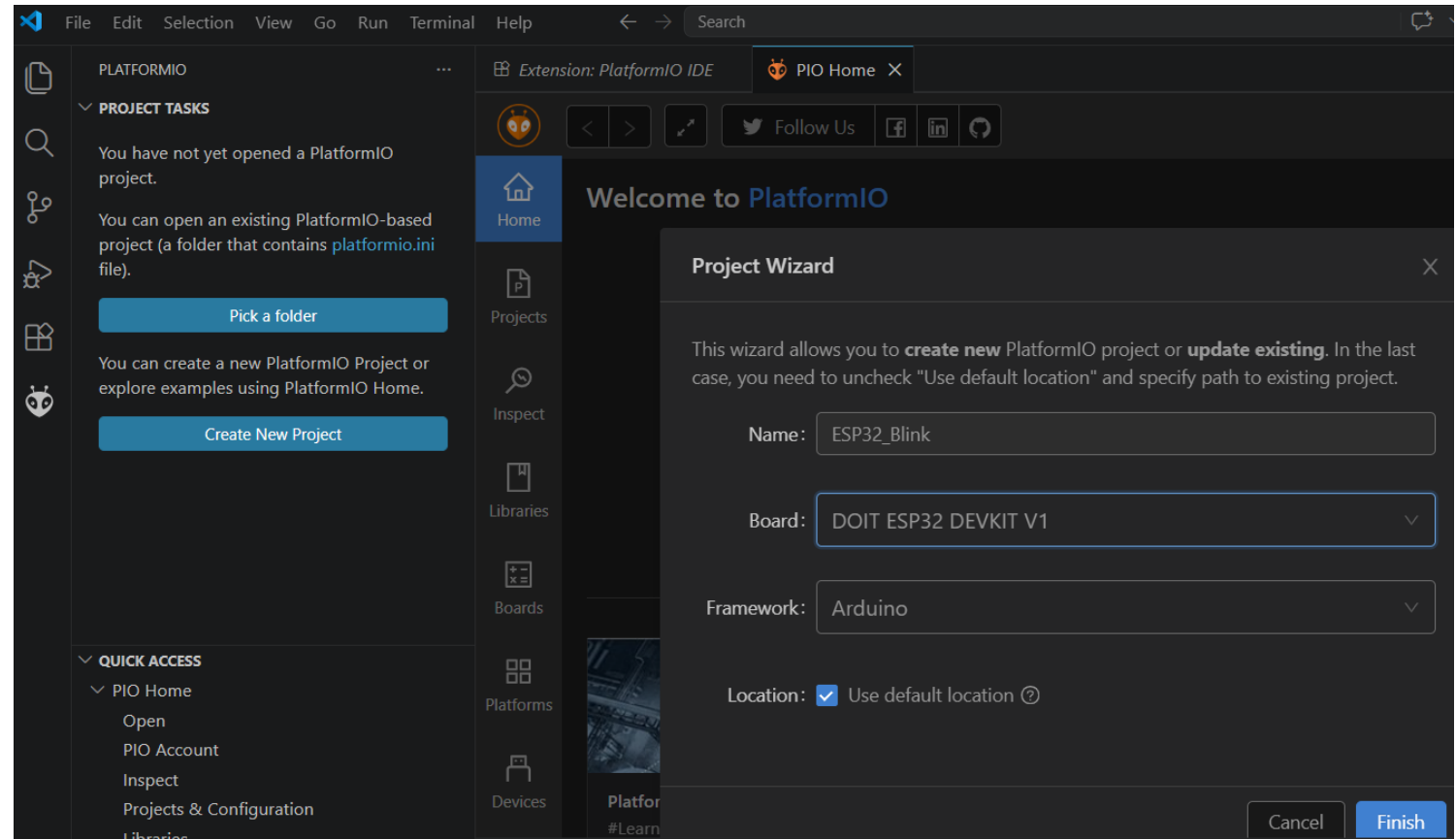
- **Install Visual Studio Code**
- **Install PlatformIO:**
 - In VS Code: Extensions, Search: PlatformIO IDE → Install



Steps Cont.

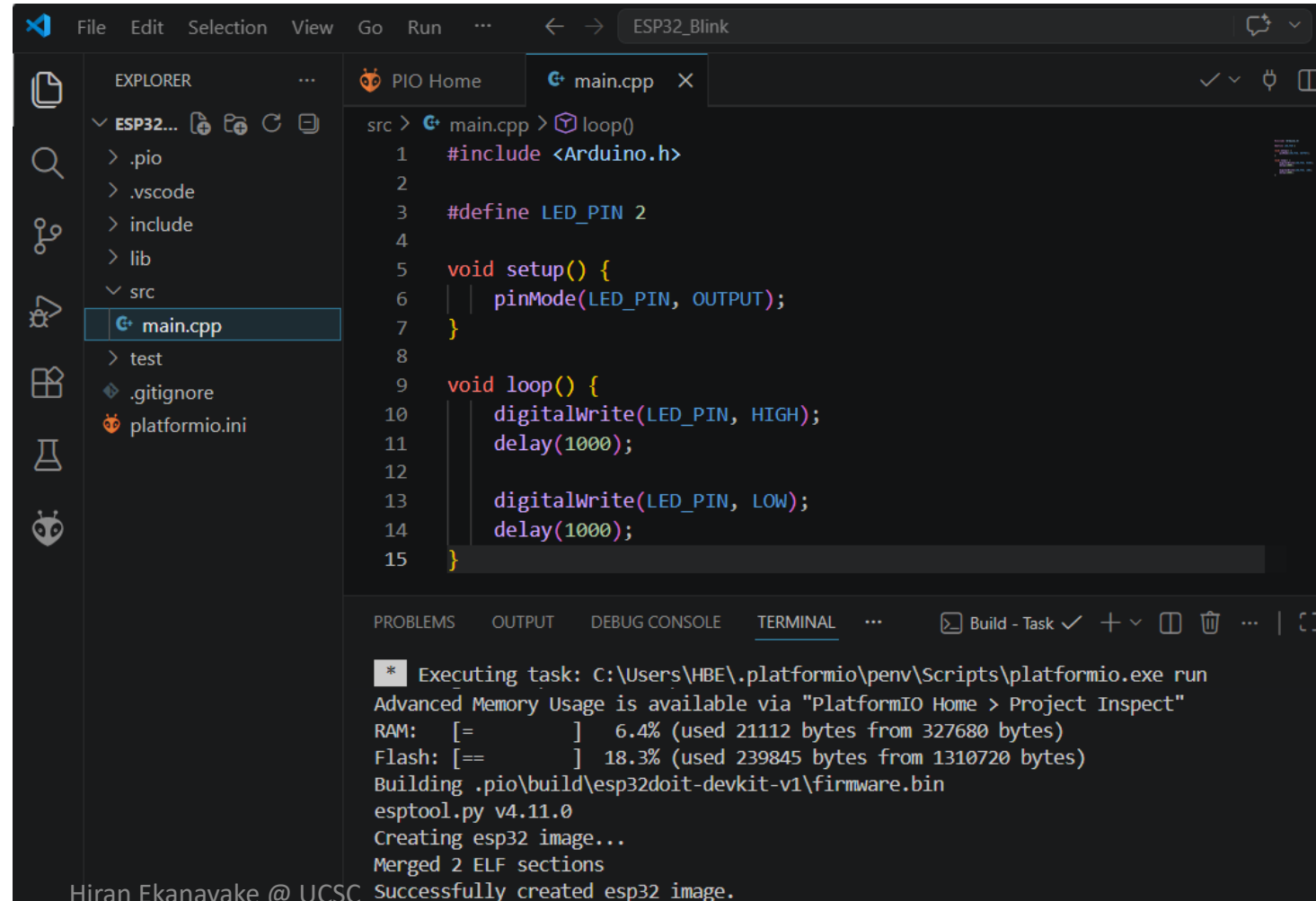
- **Create a New ESP32 Project:**

- Open: PlatformIO Home
➔ New Project
- E.g., Project Name: ESP32_Blink, Board: DOIT ESP32 DEVKIT V1, Framework: Arduino
- Click: Finish



Steps Cont.

- Write the Program:
 - Open: src/main.cpp and replace contents with the code.
- Build the Project:
 - Click: ✓ Build or PlatformIO: Build



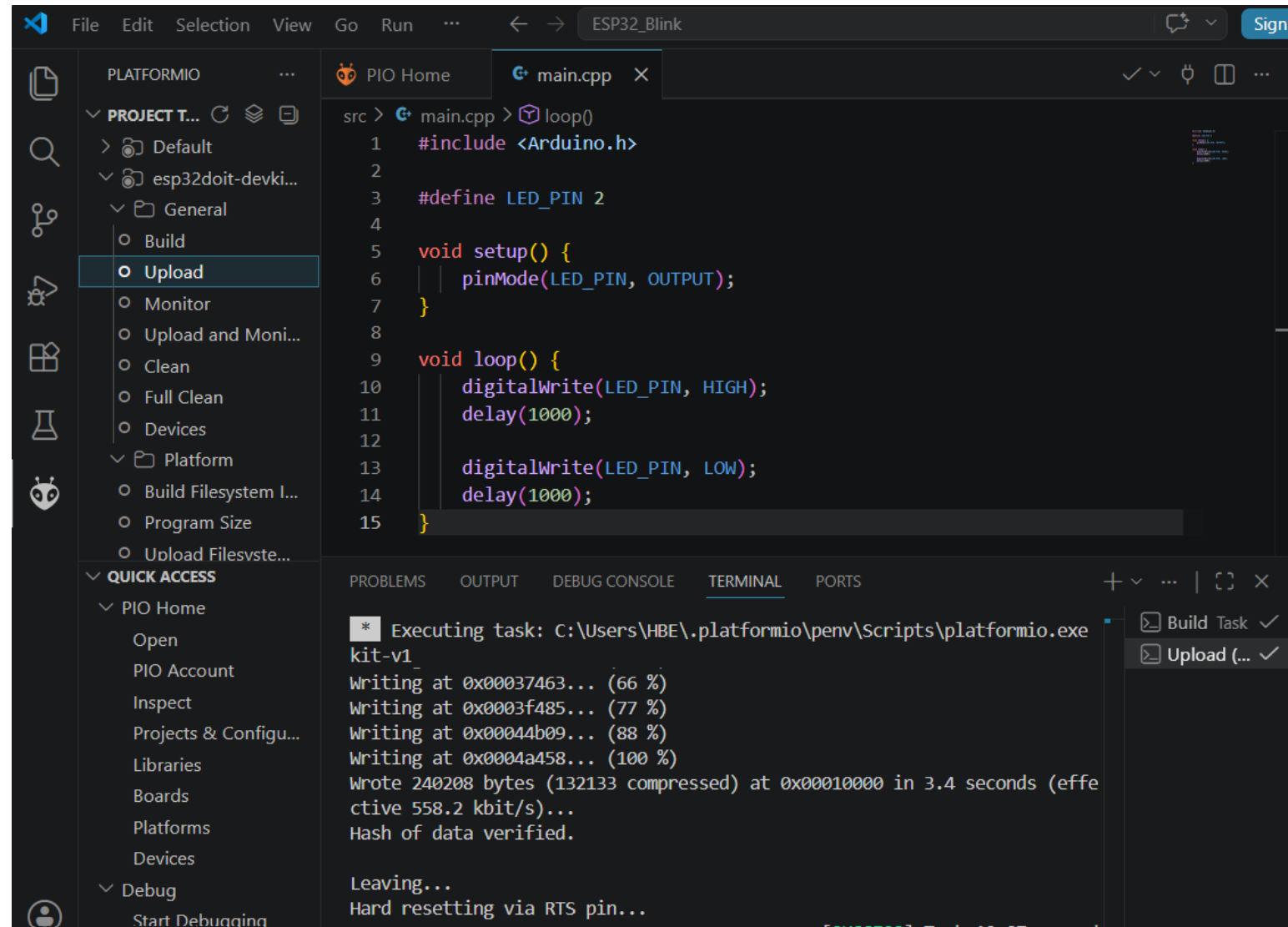
The screenshot shows the Visual Studio Code interface with a PlatformIO project. The Explorer panel on the left shows the project structure with 'main.cpp' selected in the 'src' directory. The main editor displays the code for 'main.cpp', which includes an Arduino sketch for an LED blink. The bottom panel shows the 'TERMINAL' output, which includes the command to run the task, memory usage statistics for RAM and Flash, and the successful creation of the ESP32 image.

```
src > main.cpp > loop()
1  #include <Arduino.h>
2
3  #define LED_PIN 2
4
5  void setup() {
6      pinMode(LED_PIN, OUTPUT);
7  }
8
9  void loop() {
10     digitalWrite(LED_PIN, HIGH);
11     delay(1000);
12
13     digitalWrite(LED_PIN, LOW);
14     delay(1000);
15 }
```

Executing task: C:\Users\HBE\.platformio\penv\Scripts\platformio.exe run
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [=] 6.4% (used 21112 bytes from 327680 bytes)
Flash: [==] 18.3% (used 239845 bytes from 1310720 bytes)
Building .pio\build\esp32doit-devkit-v1\firmware.bin
esptool.py v4.11.0
Creating esp32 image...
Merged 2 ELF sections
Successfully created esp32 image.

Steps Cont.

- Connect the ESP32
- Upload the Program



The platformio.ini File

- This is the heart of every PlatformIO project.
- Adding Libraries
 - Method 1: Library Manager
 - E.g., Search: Adafruit SSD1306 → Install
 - Method 2: platformio.ini
 - lib_deps = adafruit/Adafruit SSD1306
 - PlatformIO automatically downloads dependencies. This is much cleaner than manually installing libraries in Arduino IDE.

```
platformio.ini
1  ; PlatformIO Project Configuration File
2  ;
3  ; Build options: build flags, source filter
4  ; Upload options: custom upload port, speed and extra flags
5  ; Library options: dependencies, extra library storages
6  ; Advanced options: extra scripting
7  ;
8  ; Please visit documentation for the other options and examples
9  ; https://docs.platformio.org/page/projectconf.html
10
11  [env:esp32doit-devkit-v1]
12  platform = espressif32
13  board = esp32doit-devkit-v1
14  framework = arduino
15
```

Programming ESP32 DevKit V1 with **ESP-IDF** on Windows 10/11 (64-bit)

What is ESP-IDF (Espressif IoT Development Framework)?

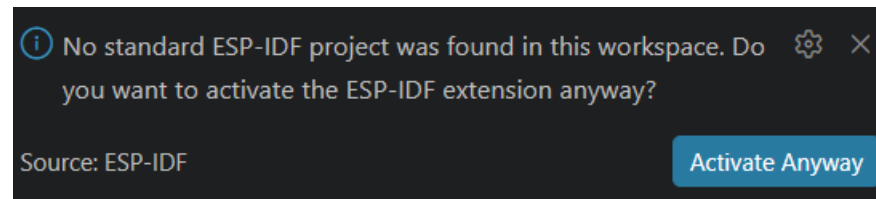
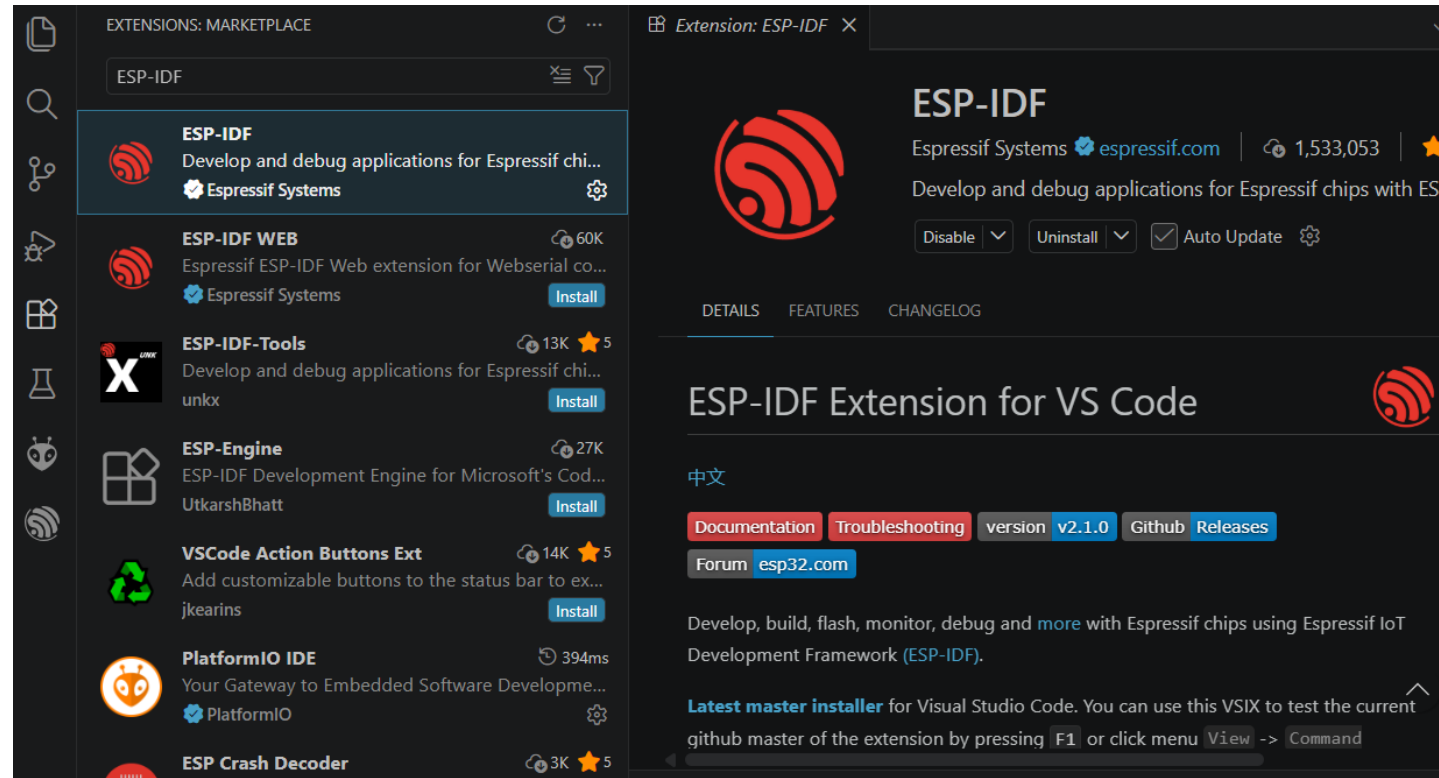
- ESP-IDF (Espressif IoT Development Framework) is the official software development framework for ESP32 devices developed by [Espressif Systems](https://www.espressif.com/).
- While Arduino simplifies programming through high-level libraries, ESP-IDF provides direct access to the ESP32 hardware and operating system features. It is the framework used by professional embedded developers and is the foundation upon which the Arduino-ESP32 core is built.

Version::	v6.0.1
Installation Path::	C:\esp
Estimated Size::	~3.5 GB
Estimated Time::	10-45 minutes

Programming ESP32 DevKit V1 with **VS Code** + **ESP-IDF** on Windows 10/11 (64-bit)

Steps

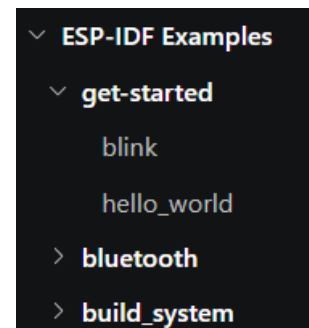
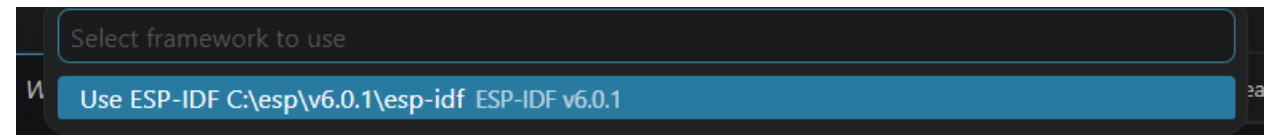
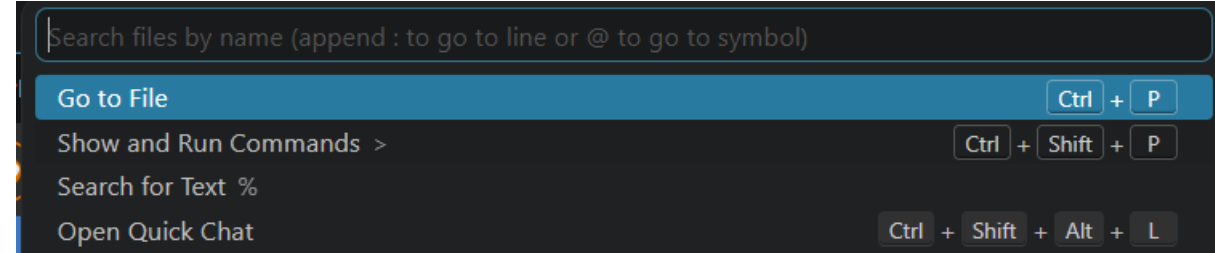
- Install Visual Studio Code
- Install ESP-IDF:
 - In VS Code: Extensions, Search: ESP-IDF → Install
- After installation, the ESP-IDF icon will appear in the left activity bar.
- Click on it and activate the extension.



Steps Cont.

- **Create a New ESP32 Project:**

- Show and Run Command (or Ctrl + Shift + P)
- Select “ESP-IDF: New Project” and then the framework to use.
- Under Examples, select “blink” or “hello_world”
- Click “Create project using template blink”



Create project using template blink							
Supported Targets	ESP32	ESP32-C2	ESP32-C3	ESP32-C5	ESP32-C6	ESP32-C61	ESP32-H2

Project Name

blink

Enter Project directory

C:\Users\HBE

\blink



Choose ESP-IDF Target (IDF_TARGET)

esp32



Choose ESP-IDF Board

Custom board



Choose serial port

COM4



OpenOCD Configuration files (Relative paths to OPENOCD_SCRIPTS)

Add files separated by comma like `interface/ftdi/esp32_devkitj_v1.cfg, board/esp32-wrover.cfg`

board/esp32-wrover-kit-3.3v.cfg

Add your ESP-IDF Component directory

Enter component path



Create Project

Steps Cont.

- Check the Program
 - “blink_example_main.c”
- Set the Configurations:
 - ESP-IDF: SDK Configuration Editor
 - Wi-Fi settings, Flash size, Partition tables, Bluetooth, FreeRTOS, Power management, Logging levels, etc.
- Build the Project:
 - ESP-IDF: Build Your Project
- Flash the ESP32:
 - ESP-IDF: Flash Your Project
 - Select: UART

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "led_strip.h"
#include "sdkconfig.h"

static const char *TAG = "example";

/* Use project configuration menu (idf.py menuconfig) to choose the GPIO to blink,
| or you can edit the following line and set a number here.
*/
#define BLINK_GPIO CONFIG_BLINK_GPIO

static uint8_t s_led_state = 0;
```

>SDK

ESP-IDF: Configure Project **SDK**Config for Coverage

ESP-IDF: Dispose of Current **SDK** Configuration Editor Server Process

ESP-IDF: ESP-IDF: Save Default **SDK**CONFIG File (save-defconfig)

ESP-IDF: **SDK** Configuration Editor (Menuconfig)

Example Configuration

Blink LED type ⓘ

GPIO

Blink GPIO number ⓘ

2

Blink period in ms ⓘ

1000

Custom Minimum Code

hello_world

```
#include <stdio.h>

void app_main(void)
{
    printf("Hello ESP32\n");
}
```

ESP-IDF: Monitor Device

```
I (266) main_task: Started
I (276) main_task: Calling
Hello ESP32
I (276) main task: Return
```

blink

```
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#define LED GPIO_NUM_2

void app_main(void)
{
    gpio_reset_pin(LED);
    gpio_set_direction(LED, GPIO_MODE_OUTPUT);

    while (1)
    {
        gpio_set_level(LED, 1);
        vTaskDelay(pdMS_TO_TICKS(1000));

        gpio_set_level(LED, 0);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

Programming ESP32 DevKit V1 with **MicroPython** on Windows 10/11 (64-bit)

Why Use MicroPython?

- **Advantages:** Easy to learn, Python syntax, Interactive development, Rapid prototyping, No compilation step
- **Disadvantages:** Slower than C/C++, Higher memory usage, Limited support for some advanced peripherals, Not ideal for hard real-time applications

Steps

- Install MicroPython Firmware:
 - Download firmware (e.g., ESP32_GENERIC-20260406-v1.28.0.bin) from https://micropython.org/download/ESP32_GENERIC/
- Install a Development Tool:
 - Recommended: [Thonny](#) (v5.0.0 by June 2026) – comes with Python 3.14
 - Other options: VS Code + PyMakr, uPyCraft, Command Line
- Flash MicroPython to ESP32:
 - To verify python, Open Command Prompt: `python --version`
 - Type: `pip install esptool` or `python -m pip install esptool`
 - To verify esptool: `esptool.py version` or `python -m esptool version`
 - Erase existing flash: `python -m esptool --chip esp32 --port COM4 erase_flash`
 - Flash MicroPython:

Steps: Flash MicroPython to ESP32

- To verify python, Open Command Prompt: `python --version`
- Type: `pip install esptool` or `python -m pip install esptool`
- To verify esptool: `esptool.py version` or `python -m esptool version`
- Erase existing flash: `python -m esptool --chip esp32 --port COM4 erase_flash`
- Flash MicroPython: `python -m esptool --chip esp32 --port COM4 --baud 460800 write_flash -z 0x1000 ESP32_GENERIC-20260406-v1.28.0.bin`
- Reboot the ESP32, press EN (Reset)

```
PS C:\Users\HBE> python -m esptool version
esptool v5.3.0
5.3.0
PS C:\Users\HBE> python -m esptool --chip esp32 --port COM4 erase_flash
Warning: Deprecated: Command 'erase_flash' is deprecated. Use 'erase-flash' instead.
esptool v5.3.0
Connected to ESP32 on COM4:
Chip type:          ESP32-D0WD-V3 (revision v3.1)
Features:           Wi-Fi, BT, Dual Core + LP Core, 240MHz, Vref calibration in eFuse, Coding Scheme None
Crystal frequency: 40MHz
MAC:                a0:b7:65:23:c3:a0

Stub flasher running.

Flash memory erased successfully in 5.7 seconds.
```

```
PS C:\Users\HBE> python -m esptool --chip esp32 --port COM4 --baud 460800 write_flash -z 0x1000 ESP32_GENERIC-20260406-v1.28.0.
bin
Warning: Deprecated: Command 'write_flash' is deprecated. Use 'write-flash' instead.
esptool v5.3.0
Connected to ESP32 on COM4:
Chip type:          ESP32-D0WD-V3 (revision v3.1)
Features:           Wi-Fi, BT, Dual Core + LP Core, 240MHz, Vref calibration in eFuse, Coding Scheme None
Crystal frequency: 40MHz
MAC:                a0:b7:65:23:c3:a0

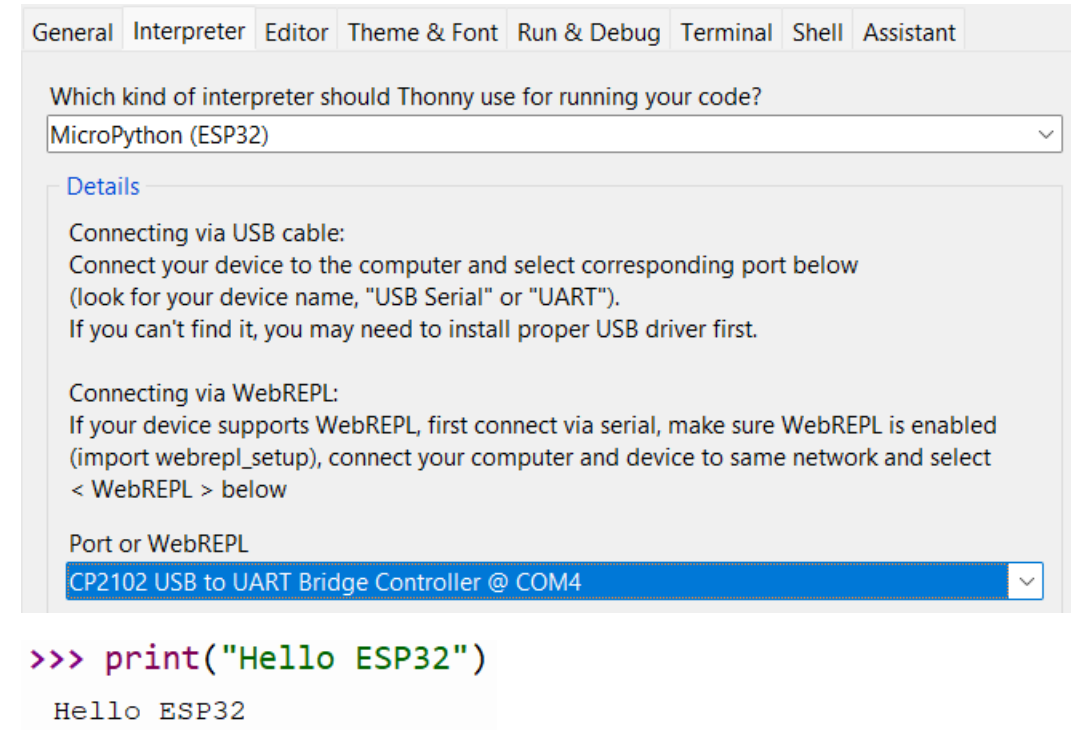
Stub flasher running.
Changing baud rate to 460800...
Changed.

Configuring flash size...
Flash will be erased from 0x00001000 to 0x001aefff...
Wrote 1760192 bytes (1152806 compressed) at 0x00001000 in 29.4 seconds (478.6 kbit/s).
Hash of data verified.

Hard resetting via RTS pin...
```

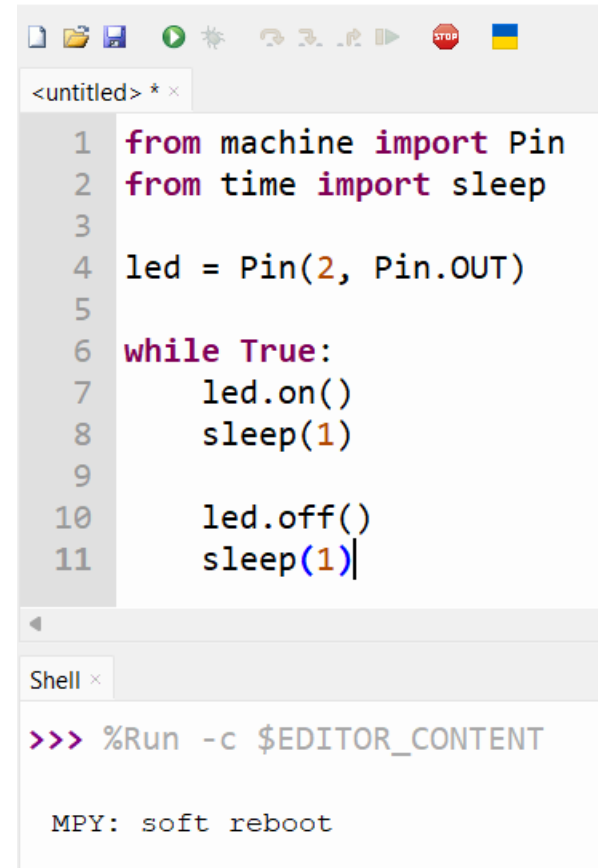
Steps Cont.

- Verify the Installation:
 - In Thonny: Tools → Options → Interpreter → MicroPython (ESP32)
 - In the shell type: `print("Hello ESP32")`
- Note: Uploading a sketch from Arduino IDE will overwrite the MicroPython firmware. Use Espressif Flash Download Tool to recover a damaged board if something goes wrong.



Steps Cont.

- First Program: Blink LED
 - Write the program
 - Press Run or F5
- File Structure (in ESP32):
 - First: boot.py – used for Wi-Fi setup and System initialization
 - Then: main.py – Contains application code.
 - To check, in shell: `import os` → `os.listdir()`



```
<untitled> * x
1 from machine import Pin
2 from time import sleep
3
4 led = Pin(2, Pin.OUT)
5
6 while True:
7     led.on()
8     sleep(1)
9
10    led.off()
11    sleep(1)|
```

```
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
```

Comparison

Feature	MicroPython	Arduino IDE	VS Code + PlatformIO	VS Code + ESP-IDF
Language	Python	C++	C++	C/C++
Learning Curve	Very Low	Low	Medium	High
Installation	Easy	Easy	Medium	Medium
Compilation Required	No	Yes	Yes	Yes
REPL Support	Yes	No	No	No
Performance	Lowest	High	High	Highest
Hardware Access	Basic	Good	Good	Full
FreeRTOS Access	Limited	Partial	Partial	Full
Debugging	Basic	Limited	Good	Excellent
Project Management	Basic	Basic	Excellent	Excellent
Unit Testing	No	No	Yes	Yes
Industry Use	Low	Moderate	High	Very High

1. **Arduino IDE** as the primary teaching environment.
2. **PlatformIO** for project work and advanced students.
3. **ESP-IDF** for demonstrating RTOS, networking internals, and industrial development.
4. **MicroPython** as an optional introductory pathway for students who already know Python.

Simulators

ESP32 Using Wokwi

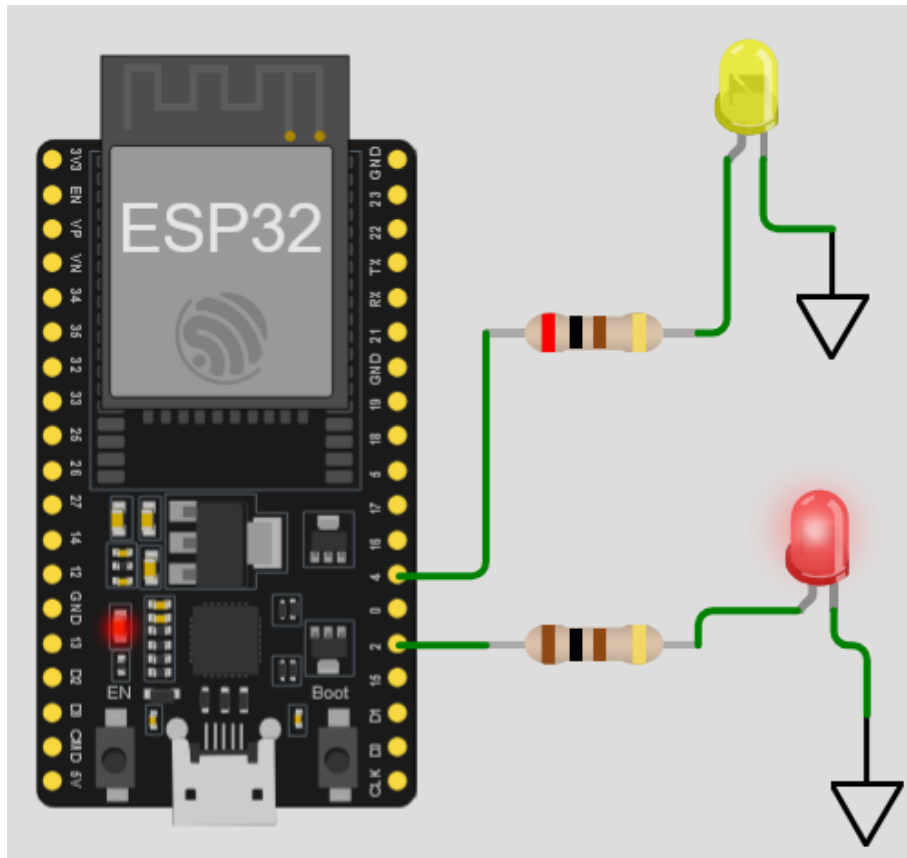
<https://wokwi.com/>

- Many boards are supported.
- Programming Languages: Arduino, ESP-IDF, MicroPython, CircuitPython
- Features: Browser-based (no installation), Drag-and-drop circuit design, LEDs, buttons, sensors, OLED/LCD displays, motors, buzzers, Serial Monitor, Logic Analyzer, GDB debugging, Virtual Wi-Fi for HTTP, MQTT, NTP, and other networking experiments.
- Limitations: Not every peripheral is simulated. Analog behavior is simplified compared to real hardware.
- **Compilation delay in the free plan.**

ESP32 Using Wokwi Example

<https://wokwi.com/projects/467908269431236609>

Multi-tasking LEDs



Hiran Ekanayake @ UCSC

```
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <driver/gpio.h>
```

```
#define LED1 GPIO_NUM_2
#define LED2 GPIO_NUM_4
```

```
void led1_task(void *pvParameters){
    while (1) {
        gpio_set_level(LED1, 1);
        vTaskDelay(pdMS_TO_TICKS(1000));
        gpio_set_level(LED1, 0);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```
void led2_task(void *pvParameters){
    while (1) {
        gpio_set_level(LED2, 1);
        vTaskDelay(pdMS_TO_TICKS(2300));
        gpio_set_level(LED2, 0);
        vTaskDelay(pdMS_TO_TICKS(2300));
    }
}
```

```
void app_main(void){
    gpio_reset_pin(LED1);
    gpio_reset_pin(LED2);

    gpio_set_direction(LED1, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED2, GPIO_MODE_OUTPUT);

    xTaskCreate(led1_task, "LED1 Task", 2048, NULL, 1, NULL);
    xTaskCreate(led2_task, "LED2 Task", 2048, NULL, 1, NULL);
}
```

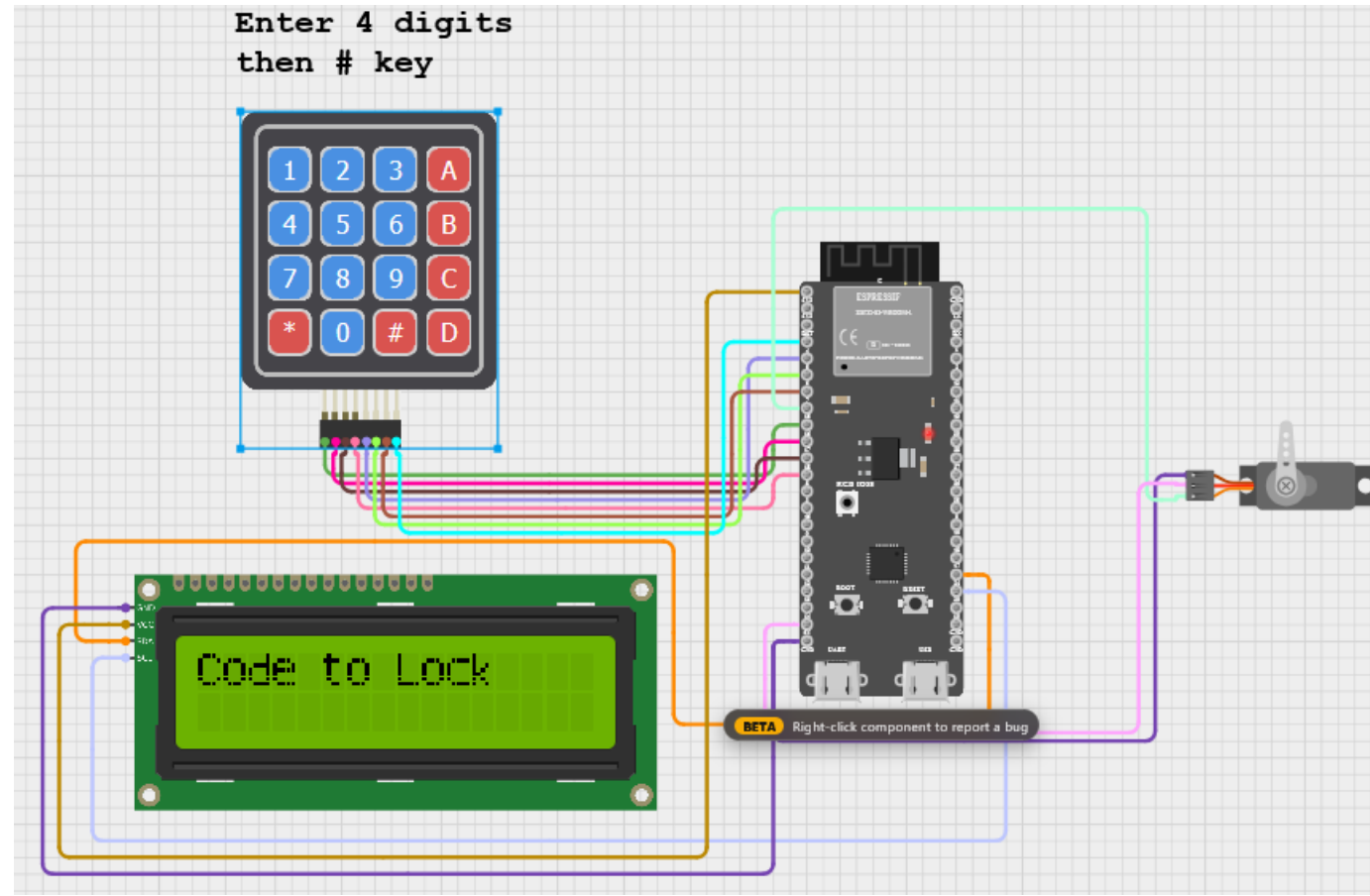
ESP32 Using Cirkuit Designer

<https://www.cirkitdesigner.com/>

- Features: Browser-based, Visual circuit editor, ESP32-S3 simulation, AI-assisted wiring and code generation (free tier available).
- Limitations: ESP32 support is not as mature or extensive as Wokwi's.

ESP32 Using Cirkit Designer Example

<https://app.cirkitdesigner.com/project/b6be54fa-7669-46eb-be47-9345e9f43241>



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <ESP32Servo.h>

// ===== Pin configuration (ESP32-S3) =====
#define I2C_SDA 21
#define I2C_SCL 20

// Keypad pins: order must be {R1, R2, R3, R4} and {C1, C2, C3, C4}
const byte ROWS = 4;
const byte COLS = 4;
byte rowPins[ROWS] = {16, 17, 18, 8}; // R1, R2, R3, R4
byte colPins[COLS] = {5, 6, 7, 4};    // C1, C2, C3, C4

// Servo
const int servoPin = 15;                // Any PWM-capable GPIO

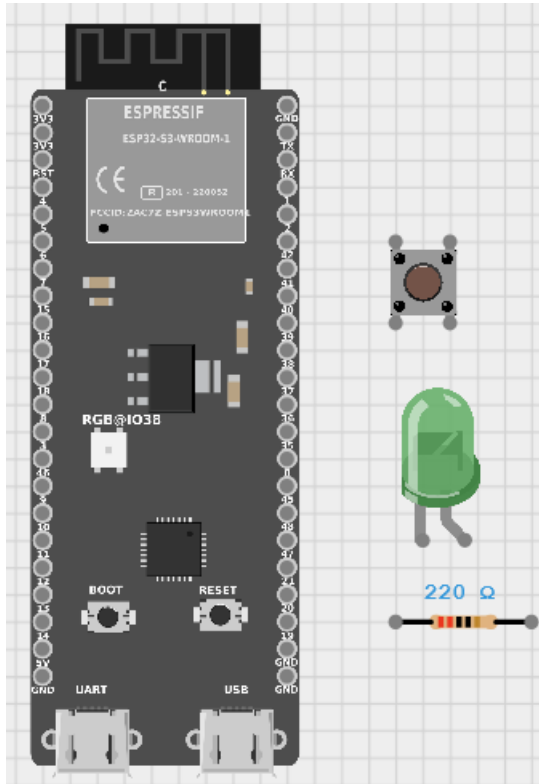
// ===== LCD =====
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ===== Keypad map =====
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

Keypad keypad = Keypad(makeKeypad(keys), rowPins, colPins, ROWS, COLS);
```

ESP32 Using Cirkit Designer & AI

Add the required components



Issue the required prompt

You

Create an ESP32 DevKit V1 circuit with one push button connected to GPIO 4 using the internal pull-up resistor. Connect an LED to GPIO 2. Generate Arduino code so that pressing the button turns the LED on and releasing it turns the LED off.

Cirkit

- ✓ Circuit wiring updated.
- **Pushbutton:** one terminal to **ESP32 GPIO 4**, the other terminal to **GND** (handled as **active LOW** using the ESP32's internal pull-up in code).
- **LED:** **GPIO 2** → **220Ω resistor** → **LED anode**, LED cathode to **GND**.

✓ Project code updated (Arduino sketch) and

It renders the design and code

```
const int BUTTON_PIN = 4; // connect button to GPIO 4
const int LED_PIN = 2;    // connect LED to GPIO 2
```

```
// Debounce settings
const unsigned long DEBOUNCE_MS = 100;
```

```
int lastReading = HIGH;
int stableState = HIGH;
unsigned long lastBounceTime = 0;
```

```
void setup() {
  // Optional: initialize serial
  Serial.begin(115200);
  delay(10);
  Serial.println("Setup: BUTTON_PIN = " + String(BUTTON_PIN) + ", LED_PIN = " + String(LED_PIN));
}
```

```
// Configure pins
pinMode(BUTTON_PIN, INPUT_PULLUP); // enable internal pull-up; button will read HIGH when not pressed
pinMode(LED_PIN, OUTPUT);
```

